

Actividad de Aprendizaje 13. El Árbol Binario de Búsqueda, implementación dinámica

Introducción

La verdad es que el profe siento que nos explicó bien el tema, se me facilitó muchísimo practicando primero por mi parte antes de empezar a programarlo.

Si tuve que checar los videos del profe pero nada más.

nodo.hpp

```
#ifndef NODO_HPP
#define NODO_HPP

class Nodo {
private:
    int dato;
    Nodo* izq;
    Nodo* der;

public:
    Nodo(int d);

    int getDato() const;

    Nodo*& getIzq();
    Nodo*& getDer();
};

#endif
```

nodo.cpp

```
#include "nodo.hpp"

Nodo::Nodo(int d) : dato(d), izq(nullptr), der(nullptr) {}

int Nodo::getDato() const {
    return dato;
}

Nodo*& Nodo::getIzq() {
    return izq;
}

Nodo*& Nodo::getDer() {
    return der;
}
```

arbol.hpp

```
#ifndef ARBOL_HPP
#define ARBOL_HPP

#include <iostream>
```

```

#include "nodo.hpp"

class Arbol {
private:
    Nodo* raiz;

    void inserta(Nodo*& r, int d);
    void preOrder(Nodo* r) const;
    void inOrder(Nodo* r) const;
    void postOrder(Nodo* r) const;
    int altura(Nodo* r) const;

public:
    Arbol();
    ~Arbol();

    void inserta(int d);

    void preOrder() const;
    void inOrder() const;
    void postOrder() const;

    int menor() const;
    int mayor() const;

    int alturaIzq() const;
    int alturaDer() const;
};

#endif

```

arbol.cpp

```

#include "arbol.hpp"

using namespace std;

Arbol::Arbol() : raiz(nullptr) {}

Arbol::~~Arbol() {}

void Arbol::inserta(int d) {
    inserta(raiz, d);
}

```

```

void Arbol::inserta(Nodo*& r, int d) {
    if (r == nullptr) {
        r = new Nodo(d);
        return;
    }

    if (d < r->getDato()) {
        inserta(r->getIzq(), d);
    } else {
        inserta(r->getDer(), d);
    }
}

void Arbol::preOrder(Nodo* r) const {
    if (r == nullptr)
        return;
    cout << r->getDato() << " ";
    preOrder(r->getIzq());
    preOrder(r->getDer());
}

void Arbol::inOrder(Nodo* r) const {
    if (r == nullptr)
        return;
    inOrder(r->getIzq());
    cout << r->getDato() << " ";
    inOrder(r->getDer());
}

void Arbol::postOrder(Nodo* r) const {
    if (r == nullptr)
        return;
    postOrder(r->getIzq());
    postOrder(r->getDer());
    cout << r->getDato() << " ";
}

void Arbol::preOrder() const {
    preOrder(raiz);
    cout << endl;
}

```

```

void Arbol::inOrder() const {
    inOrder(raiz);
    cout << endl;
}

void Arbol::postOrder() const {
    postOrder(raiz);
    cout << endl;
}

int Arbol::menor() const {
    Nodo* aux = raiz;
    while (aux && aux->getIzq() != nullptr) {
        aux = aux->getIzq();
    }
    return aux ? aux->getDato() : -1;
}

int Arbol::mayor() const {
    Nodo* aux = raiz;
    while (aux && aux->getDer() != nullptr) {
        aux = aux->getDer();
    }
    return aux ? aux->getDato() : -1;
}

int Arbol::altura(Nodo* r) const {
    if (r == nullptr)
        return 0;

    int izq = altura(r->getIzq());
    int der = altura(r->getDer());

    return 1 + (izq > der ? izq : der);
}

int Arbol::alturaIzq() const {
    if (raiz == nullptr)
        return 0;
    return altura(raiz->getIzq());
}

int Arbol::alturaDer() const {

```

```

    if (raiz == nullptr)
        return 0;
    return altura(raiz->getDer());
}

```

main.cpp

```

#include <cstdlib>
#include <ctime>
#include <iostream>
#include "arbol.hpp"

using namespace std;

int main() {
    Arbol arbol;
    int N;

    cout << "Ingrese N: ";
    cin >> N;

    srand(time(0));

    for (int i = 0; i < N; i++) {
        int num = rand() % 100001;
        cout << num << endl;
        arbol.inserta(num);
    }

    cout << "\nPreOrder:\n";
    arbol.preOrder();

    cout << "\nInOrder:\n";
    arbol.inOrder();

    cout << "\nPostOrder:\n";
    arbol.postOrder();

    cout << "\nMenor: " << arbol.menor() << endl;
    cout << "Mayor: " << arbol.mayor() << endl;

    cout << "Altura izquierda: " << arbol.alturaIzq() << endl;
    cout << "Altura derecha: " << arbol.alturaDer() << endl;
}

```

```
    return 0;  
}
```

```
C:\Windows\system32\cmd.e: X + v  
Ingrese N: 5  
10436  
16269  
28821  
28804  
7699  
  
PreOrder:  
10436 7699 16269 28821 28804  
  
InOrder:  
7699 10436 16269 28804 28821  
  
PostOrder:  
7699 28804 28821 16269 10436  
  
Menor: 7699  
Mayor: 28821  
Altura izquierda: 1  
Altura derecha: 3  
  
C:\programacion_cpp>
```

```
C:\Windows\system32\cmd.e: X + v  
Ingrese N: 15  
10492  
2384  
4832  
11896  
19206  
7879  
246  
29960  
28705  
17240  
23777  
29865  
26198  
29617  
15401  
  
PreOrder:  
10492 2384 246 4832 7879 11896 19206 17240 15401 29960 28705 23777 26198 29865 29617  
  
InOrder:  
246 2384 4832 7879 10492 11896 15401 17240 19206 23777 26198 28705 29617 29865 29960  
  
PostOrder:  
246 7879 4832 2384 15401 17240 26198 23777 29617 29865 28705 29960 19206 11896 10492  
  
Menor: 246  
Mayor: 29960  
Altura izquierda: 3  
Altura derecha: 6
```

```
C:\Windows\system32\cmd.e x + v
Ingrese N: 1000
10547
21267
13611
27757
30714
9097
20390
12965
14490
11027
21263
30614
9343
15993
32220
2417
29800
2449
2653
16340
19912
22287
10050
8769
8044
2719
22878
21200
12639
```

```
PreOrder:
10547 9097 2417 305 285 168 88 106 111 240 228 858 305 588 443 325 445 496 757 633 626 596 642 825 810 810 1387 1224 114
6 1064 1032 986 861 874 937 1016 1032 1072 1136 1111 1168 1155 1169 1298 1278 1320 1310 1330 1324 1528 1413 1388 1389 14
28 1473 1578 1536 1979 1918 1827 1771 1766 1734 1664 1579 1625 1712 1825 1776 1934 1926 2183 2164 2126 2028 2010 2010 21
80 2413 2195 2184 2240 2200 2242 2329 2449 2434 2426 2653 2455 2583 2545 2458 2508 2579 8769 8044 2719 2697 4384 4285 36
81 3247 3199 2921 2793 2907 2805 2918 2983 3174 3097 3018 3014 3033 3167 3097 3197 3206 3199 3218 3589 3316 3298 3271 34
52 3338 3398 3343 3386 3877 3873 3733 3710 3835 4079 3939 3914 4074 4047 4034 4002 4105 4188 4170 4141 4230 4370 4310 43
45 4316 6816 6667 6263 4576 4483 4388 4429 4524 5145 4912 4742 4799 4744 5137 5129 4944 4914 5006 5008 5699 5642 5249 51
54 5155 5570 5299 5364 5694 5690 6133 5735 5725 5708 5918 5879 5779 5759 5777 5966 6159 6204 6611 6427 6331 6323 6273 63
28 6340 6579 6494 6463 6490 6544 6524 6510 6648 6716 6702 6681 7657 6856 6848 6818 7487 7292 7089 6903 6887 6945 6959 69
50 7071 7033 7238 7124 7164 7215 7195 7399 7348 7317 7387 7453 7404 7451 7457 7544 7543 7567 7990 7757 7738 7689 7681 77
30 7768 7803 7789 7866 7896 7988 8318 8186 8063 8052 8123 8097 8077 8158 8158 8160 8273 8221 8627 8506 8490 8466 8478 85
00 8493 8617 8514 8552 8580 8570 8583 8633 8627 9045 8816 8806 8998 8831 8847 8915 9060 9064 9070 9343 9267 9126 9120 91
31 9240 9153 9215 9168 9178 9299 9275 9311 10050 9844 9758 9701 9512 9423 9410 9468 9454 9777 9776 9762 9800 10032 9912
9886 10026 10523 10067 10058 10266 10175 10131 10079 10103 10082 10210 10207 10338 10330 10377 10367 10501 10475 10440 1
0477 10541 21267 13611 12965 11027 10922 10598 10549 10593 10580 10812 10752 10670 10625 10668 10754 10833 10833 10853 1
0991 10990 10992 12639 11450 11252 11193 11038 11029 11112 11203 11327 11331 11335 11419 11406 12259 11614 11601 11575 1
1844 11744 11728 11618 11838 11748 11841 12160 11854 11936 11868 12121 11951 12065 12047 12110 12110 12144 12257 12248 1
2161 12181 12542 12292 12267 12373 12331 12379 12491 12674 12880 12860 12831 12688 12821 12856 12842 12948 12932 12885 1
2906 12945 13075 13016 13171 13108 13129 13522 13191 13415 13218 13354 13472 13499 20390 14490 13917 13719 13787 13720 1
3761 13802 13851 14231 14192 13948 13933 14096 14019 13991 14019 14089 14168 14201 14471 14343 14270 14238 14311 14273 1
4331 14485 14475 15993 15166 14711 14634 14495 14818 15028 14885 14854 15010 14977 14964 14996 14989 15016 15094 15029 1
5091 15426 15410 15354 15327 15253 15183 15247 15276 15331 15421 15410 15572 15535 15511 15577 15576 15660 15609 15609 1
5809 15723 15761 15954 15988 16340 16023 16171 16065 16080 16198 19912 19348 19085 18565 18314 18110 17292 16772 16524 1
6722 16714 16550 16561 16608 16711 16751 16727 16765 17038 16993 16972 16960 16822 16857 16853 16935 16898 17060 17048 1
7041 17109 17105 17271 17176 17584 17371 17313 17459 17422 17446 17572 17576 17716 17681 17626 17991 17933 17908 17881 1
7830 17803 17917 18059 18046 18270 18158 18165 18198 18191 18401 18527 18465 18420 18437 18502 18901 18670 18624 18787 1
8868 18843 18891 18996 18910 19001 19215 19141 19115 19140 19211 19283 19319 19299 19340 19803 19503 19406 19352 19363 1
9354 19488 19588 19536 19759 19597 19632 19748 19785 19781 19818 19888 19973 19966 20241 20215 20119 20113 20049 20006 2
```

```
Menor: 88
Mayor: 32695
Altura izquierda: 21
Altura derecha: 22
}
```

Conclusión

Fue un tema divertido, me gustó, pero sin más. La verdad es que no me gusta mucho trabajar con punteros y espero no seguirlos utilizando, ya que me enamoré de trabajar con listas estáticas, o arreglos de objetos pero bueno, si es lo que hay me adapto.

